



北京工商大学
BEIJING TECHNOLOGY AND BUSINESS UNIVERSITY

操作系统跨特权级 统一调试平台

面向 Rust 操作系统的跨特权级与异步统一调试

团队成员：曾小红 · 王浩铭 · 武雪妍 指导教师：吴竞邦

北京工商大学 · 计算机与人工智能学院 · 2026 年 6 月

项目目录

CONTENTS

01 项目背景

操作系统调试与异步调试的核心挑战

02 核心目标与完成情况

核心目标与完成度总览

03 核心工作分述

四个核心技术方案

04 测试与验证

StarryOS 与 embassy 双验证

05 决赛计划

待解决的问题

项目背景：赛题与项目增量

1 项目背景



赛题

proj55-源代码级内核调试器

2 核心目标与完成情况



历届完成阶段

➤2023年：针对**Rust语言同步函数**调试以及**跨特权级断点**调试

成员：陈志扬、向驹韬

➤2024年：针对**C语言**进行扩充以及实际开发板**硬件调试**的支持

成员：杨雅琪、张弈帆、丛湘纹

➤2025年：针对**Rust语言异步函数**的跟踪调试进行设计和开发

成员：曾小红、张弈帆、董嘉誉

3 核心工作分述

4 测试与验证



今年

5 决赛计划

➤ 针对 **StarryOS + 异步操作系统** 进行调试适配，并实现界面展示 ◀

项目背景：操作系统调试的核心困难

1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划

困难一 特权级切换导致符号表断裂

- 内核态与用户态使用不同地址空间和符号表
- 特权级切换时调试器须同步切换符号表
- 操作系统启动过程发生数十次切换，手动操作不可行

困难二 前序机制在组件化 OS 上失效

- ✓ 前序工作在 rCore 上验证了四状态机断点组管理机制
- ✓ 该机制依赖三个与源码组织相关的隐含前提（如下图）
- ✓ **组件化 OS（StarryOS）不满足以下前提**



项目背景：Rust 异步调试的执行流不透明

核心困难

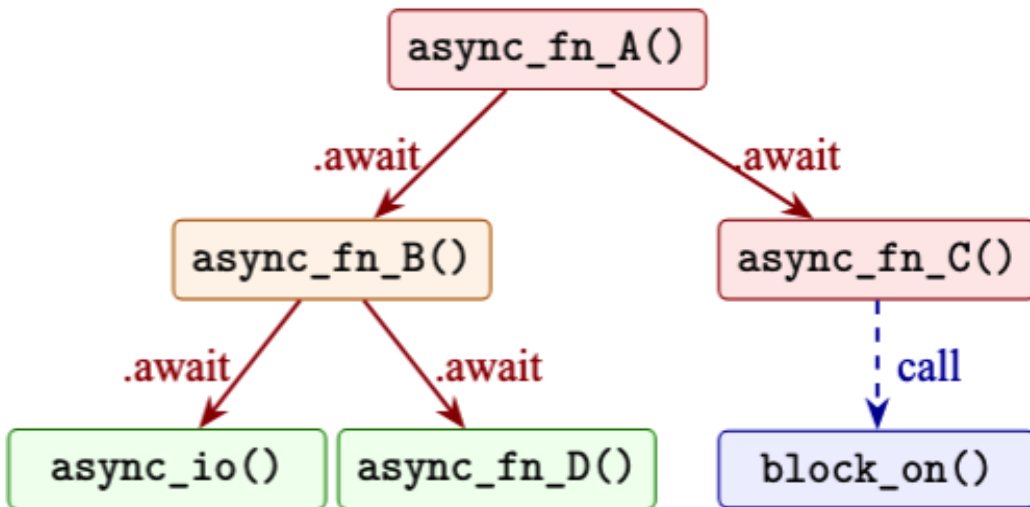
- 编译器将 async 函数拆分为离散的 poll 阶段，**逻辑执行流被切分为状态片段**
- GDB 暂停时**只能看到当前 poll 入口**，无法还原跨 poll 周期的执行因果链
- 传统调用栈**不区分同时并发的多个协程**——执行流"混在一起"，难以独立追踪

GDB 物理调用栈（单次暂停）



每次暂停只能看到当前 poll 入口的栈帧
无法还原 Future 之间的等待关系

async-debug 逻辑调用树（跨 poll 周期）



—— await edge: 源码层等待关系 (`__awaitee` 字段)

-- call edge: 机器级控制流 (call-site 跟踪)

前序基础（2025）

- 提出 await edge 和 call edge 双关系恢复方法
- 从编译后的状态机碎片中还原异步执行的逻辑依赖关系

项目总览：操作系统跨特权级统一调试平台

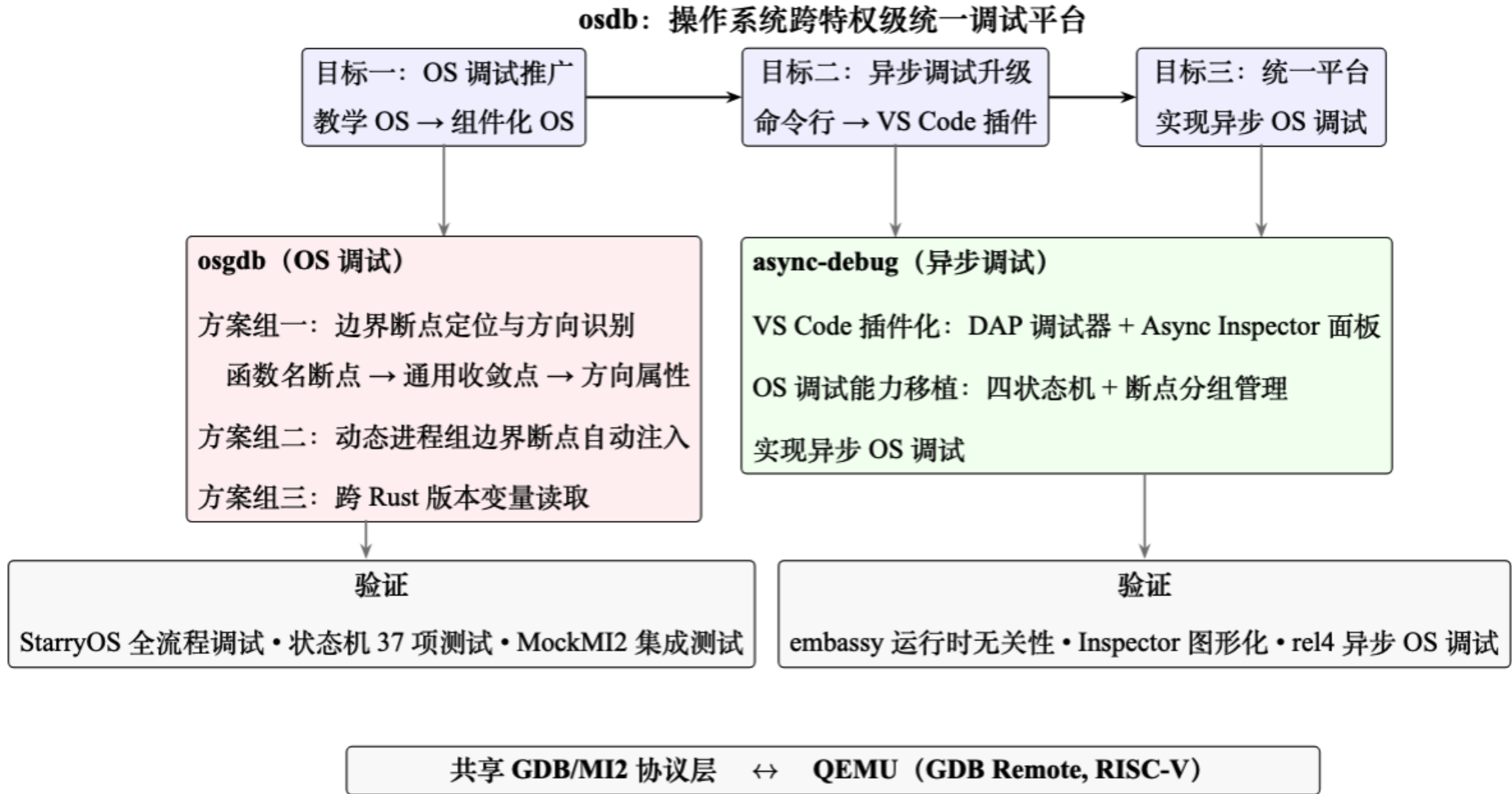
1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划



osgdb (三组方案) + async-debug (插件与面板), 共享 GDB/MI2 协议层

三大核心目标

1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划

一、适配StarryOS

从**教学OS (rCore)**
的调试推广到**组件OS (StarryOS)**

已完成

二、异步调试界面实现

从 GDB 命令行脚本
升级为 **VS Code 插件**
图形化展示

已完成

三、异步操作系统调试

合并OS调试与异步调试
实现异步操作系统调试

进行中

项目开发基于两大子系统

✓ osgdb: 特权级切换时的断点组与符号表自动管理

✓ async-debug: 异步执行流逻辑还原与图形化展示

他们共享 GDB / MI2 协议层, 通过 QEMU 连接 RISC-V 虚拟机

项目完成情况

1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划

模块	状态	验证目标
边界断点定位与方向识别	已完成	StarryOS
动态进程组边界断点自动注入	已完成	StarryOS
跨 Rust 版本变量读取	已完成	StarryOS
异步双关系恢复 (await + call edge)	已完成	embassy
VScode逻辑树构建图形界面	已完成	异步测试用例+ embassy
异步操作系统调试打通	进行中	rel4异步操作系统
调试配置自动推导	待实现	
特权级切换性能优化	待实现	

两大子系统**七项功能已完成**, StarryOS 与 embassy 平台均已验证

原OS调试设计的三条隐含前提

在 StarryOS 上全部不成立

1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划

前提	教学 OS (rCore)	组件化 OS (StarryOS)
前提一	内核与用户源码在同一工作区 文件路径可直接 定位断点 ✓	切换边界函数位于外部 crate 文件路径 归组机制失效 ✕
前提二	所有系统调用经 统一ecall入口 一个断点覆盖全部切换路径 ✓	ecall 指令 分散 在C库各封装中 用户态不存在统一汇聚点 ✕
前提三	调试前已知用户程序 断点组可提前 静态配置 ✓	交互式shell动态启动任意程序 断点组只能在 运行时创建 ✕

rCore上的设计条件在StarryOS上均不满足，边界**断点机制**需重新设计

核心工作一：边界断点的定位与方向识别

子问题拆解

函数名断点

- 外部库无本地源码无法用文件路径指定断点
- ✓ 改用函数名由符号表自动确定断点位置

通用收敛点

- 系统调用函数分散在各 C 库封装
- ✓ 将断点移至内核态唯一的syscall 分发函数

方向属性

- 特权级切换的相关断点同在内核地址空间, PC 寄存器无法区分方向
- ✓ 为边界断点引入 kernel_to_user 和 user_to_kernel 方向属性

rCore —两边界在不同地址空间
内核空间

kernel→user border
trap_return

用户空间

user→kernel border
syscall()

PC 地址即可判断方向

StarryOS —两边界同在内核空间
内核空间

kernel→user
enter_user (axhal)

user→kernel
handle_syscall

PC 无法判断 → 方向属性显式标注

通过以上三个方案解决组件化OS边界断点失效问题

1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划

核心工作二：动态进程组的边界断点自动注入

osgdb 方案组二

1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划

问题分析

- StarryOS 以交互式 shell 运行
- 用户在终端中手动敲入命令启动程序
- 新进程在 `execve` 系统调用时才动态出现
- 调试器无法预判会运行什么、何时运行
- 原调试器调试 rCore 时，是基于配置文件中的静态进程组实现的

现状

- 动态创建的进程组缺少边界断点的动态注入
- 导致特权级状态机切换链条断裂
- 调试器无法从用户态切回内核态

解决方案

配置阶段预存所有 user → kernel 方向边界断点

维护一个"待注入队列"

任意时刻动态创建进程组时，自动从队列继承边界断点

确保每个用户进程组都具备返回内核态的通道

从"静态进程调试"到"动态进程交互式调试"，保证调试器对任何进程随时可正常调试

核心工作三：跨 Rust 版本的变量读取

osgdb 方案组三

1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

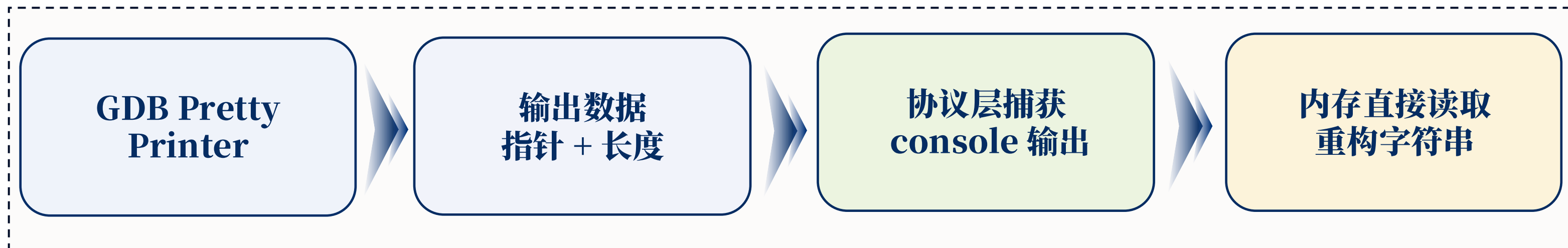
5 决赛计划

Hook断点获取下一进程的困难：

1. Rust String 类型内部字段路径随编译器版本变化
2. 原机制通过硬编码字段路径**只能兼容特定 Rust 版本**
3. 预想通过GDB输出获取字段结果，但 **GDB/MI 协议无获取命令输出的API**

解决方案

- 利用 GDB **按序处理命令的隐式保证**
- 在协议层自建控制台**输出捕获机制**
- 从 pretty printer 输出结果中提取**数据指针和长度**
- 从目标内存直接读取原始字节，**重构下一进程名字符串**



基于GDB协议顺序性捕获输出，兼容不同Rust编译器版本

针对目标二：VS Code 插件与异步调试界面

async-debug 工程化

1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划

```
(gdb) find-poll-fn
[rust-future-tracing] Finding all poll methods...
[rust-future-tracing] Using regex: ^\s*(\d+):\s+(.*)? -> core::task::poll::Poll<(.*)>;$
[rust-future-tracing] Found 8 poll methods.
[rust-future-tracing] Poll map written to: results/poll_map.json
[rust-future-tracing] Please edit this file to select the futures you want to trace.
(gdb) init-dwarf-analysis ../embassy/examples/std/target/debug/tick
Loading DWARF information from: ../embassy/examples/std/target/debug/tick
Successfully initialized DWARF analysis for: ../embassy/examples/std/target/debug/tick
Found 210 compilation units
You can now access the tree with: python tree = gdb.dwarf_tree
Or access DWARF info with: python info = gdb.dwarf_info
(gdb) start-async-debug
[rust-future-tracing] Step 1: Reading user-selected interesting functions...
[rust-future-tracing] Found interesting poll function: static fn tick::__embassy_main_task::{async_fn#0}(*mut core::ta
::Context)
[rust-future-tracing] Found interesting poll function: static fn tick::__led_task_task::{async_fn#0}(*mut core::task::wa
ext)
[rust-future-tracing] Found interesting poll function: static fn tick::__sensor_task_task::{async_fn#0}(*mut core::task:
ontext)
```

```
=====
Asynchronous Backtraces
=====
Process 16016:
Thread 16016:
  Coroutine 'ManuallyDrop<core::cell::UnsafeCell<tick::__embassy_main_task::{async_fn_env#0}>>' (ID: 58884):
    Stack is empty.
  Coroutine 'ManuallyDrop<core::cell::UnsafeCell<tick::__led_task_task::{async_fn_env#0}>>' (ID: 58854):
    Stack is empty.
  Coroutine 'ManuallyDrop<core::cell::UnsafeCell<tick::__sensor_task_task::{async_fn_env#0}>>' (ID: 58824):
    Stack is empty.
```

```
=====
Asynchronous Backtraces
=====
Process 16016:
Thread 16016:
  Coroutine 'ManuallyDrop<core::cell::UnsafeCell<tick::__embassy_main_task::{async_fn_env#0}>>' (ID: 58884):
    Stack is empty.
  Coroutine 'ManuallyDrop<core::cell::UnsafeCell<tick::__led_task_task::{async_fn_env#0}>>' (ID: 58854):
    Stack is empty.
  Coroutine 'ManuallyDrop<core::cell::UnsafeCell<tick::__sensor_task_task::{async_fn_env#0}>>' (ID: 58824):
    tick::__sensor_task_task::{async_fn_env#0}
```

原 async-debug 为命令行版本
不易操作、理解异步执行流



实现 VS Code 插件化
+
Async Inspector 面板



针对目标二：VS Code 插件与异步调试界面

async-debug 工程化

三层架构

数据层：GDB Python 脚本，输出 JSON 快照

传输层：调试适配器通过 Webview 消息通道传递数据

展示层：前端代码，渲染交互式树形图

面板核心设计

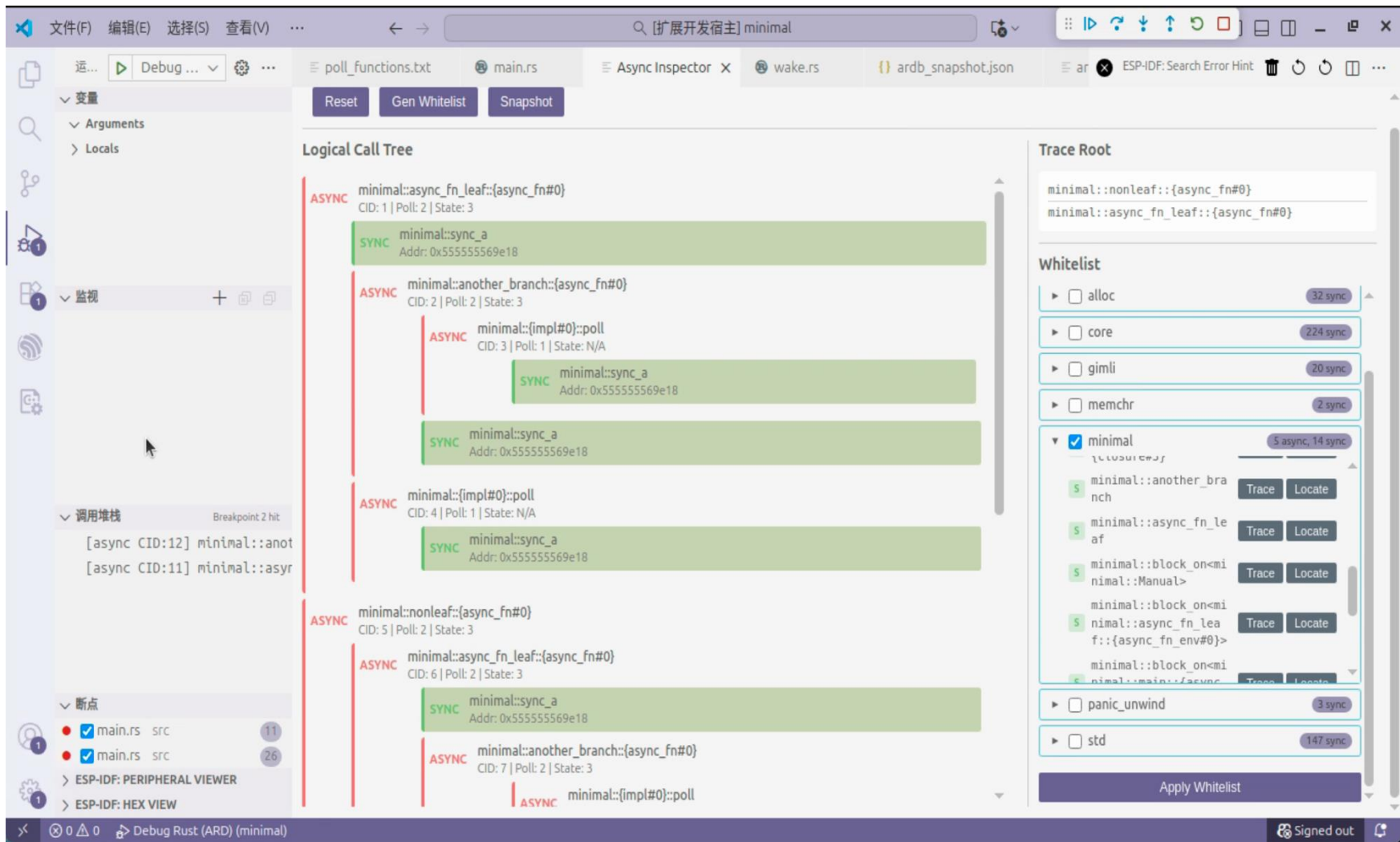
✓ 红色节点 async 协程 蓝色节点 sync

✓ 节点标注三项信息：

✓ 协程 ID poll 次数 运行状态（运行中 等待中 已完成）

✓ 点击任意节点自动跳转 VS Code 编辑器对应源码位置

✓ 调试全流程 UI 集成



测试与验证：适配 StarryOS 调试 + embassy 无运行时异步调试

1 项目背景

演示视频链接如下，可点击跳转查看：

◇ [async-debug调试演示](#)：演示程序包含3层嵌套的async 函数和手动实现的

Future，共产生11个协程实例，重在展示异步逻辑树构建效果

◇ [StarryOS调试演示](#)：展示了从启动QEMU、连接GDB、内核初始化、进入shell

、手动运行用户程序到断点组自动切换的全过程

◇ [embassy调试演示](#)：是一种 Rust 嵌入式异步框架，旨在验证异步调试运行时无

关性

◇ [rel4 调试演示](#)：rel4 是 Rust 编写的异步操作系统，作为目标三最终适配目标，

异步操作系统调试功能还在完善中，当前演示结果为初步适配成果

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划

初赛已完成工作总结

1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划

Osgdb:从教学 OS 推广到组件化 OS

- 边界断点定位与方向识别 —— 三子问题
 - 动态进程组边界断点自动注入 —— 待注入队列
 - 跨 Rust 版本变量读取 —— 协议顺序性捕获
- 验证: StarryOS 全流程 + MockMI2 37 项测试

async-debug:从命令行到 VS Code 插件

- 双关系恢复 —— await edge + call edge
 - Async Inspector 图形化面板
 - VS Code 调试器完整插件
- 验证: embassy 异步拓扑 + minimal 正确性测试

异步OS调试架构基础已就绪

- 两个子系统共享同一 GDB/MI2 协议层, 为异步 OS 调试奠定架构基础
- OS 调试能力 (四状态机、断点分组管理) 已移植至 async-debug

待解决问题与决赛计划

1 项目背景

2 核心目标与完成情况

3 核心工作分述

4 测试与验证

5 决赛计划

待解决问题	当前困难	决赛方案
异步 OS 调试打通	1. 页表切换 导致 跨地址空间 Future 状态机不可读 2. Release编译优化消除等待关系字段	1. 编译期信息导出策略 2. 编译器辅助标记替代路径 3. 已在rel4上开始适配
调试配置自动推导	1. 五类关键配置项均需手动填写，要求开发者深入理解 OS 源码与地址空间	1. 解析内核ELF符号表 2. 链接脚本自动推导 3. 至少实现三类自动推导
状态机切换性能优化	1. 由于特权级切换涉及单步机制，shell 交互场景中单字符触发约 3 次完整特权级切换，导致输入单字符延迟可达秒级	1. finish命令替代逐指令单步 2. 切换冷却期机制 + 用户断点感知策略 3. 目标：延迟降低90%以上



北京工商大学
BEIJING TECHNOLOGY AND BUSINESS UNIVERSITY

感谢各位老师 敬请批评指正

团队成员：曾小红 · 王浩铭 · 武雪妍 指导教师：吴竞邦

北京工商大学 · 计算机与人工智能学院 · 2026 年 6 月